

TestVDB: Detecting Documentation-Implementation Defects in Vector Databases via LLM-Derived Behavioral Claims

Authors TBD

tbd@example.org

TBD (anonymity depends on venue)

TBD

Abstract

Vector Database Management Systems (VDBMSs) store the embeddings that retrieval-augmented Large Language Model (LLM) applications depend on, yet most of their defects are logical bugs that produce no crash and thus escape crash-oracle fuzzers such as VDBFuzz. We target *documentation-implementation defects*: cases where a VDBMS silently accepts an input or behavior that violates its API documentation, such as accepting `nprobe=0` when the documentation declares the range as `[1, 16384]`. These defects corrupt query semantics. Across 107 issues TestVDB surfaced, maintainers acknowledged 50 as true-positive defects (69% adjudicated yield; 15 fixed via merged PR, 16 with open fix-PRs, 19 acknowledged but unfixed). Because the boundary is natural-language prose, current instantiations of deterministic oracles (differential, metamorphic, property-based) miss these accept/reject decisions, leaving an LLM as the practical oracle. We instantiate TestVDB, a four-stage pipeline (claim extraction, test generation, execution, confirmation) that uses LLMs to read documentation, generate tests, and adjudicate responses. Two failure modes produce false positives: hallucination in claim extraction and self-preference bias in judgment. A multi-perspective judging baseline raises precision but collapses recall, so we introduce a *dev-reviewer* agent that acts as a source-grounded falsifier, reproducing each candidate, cross-checking it against source, and trying to disprove it. On 48 maintainer-adjudicated candidates, the dev-reviewer reaches 67% precision and 74% recall (3-run ensemble), versus 37% recall without the source anchor. These results use a single LLM backbone (GLM-5.2); cross-family generalization is an open question. A bidirectional probe against VDBFuzz on Qdrant shows TestVDB reaches a crash-class defect by contract reasoning, while VDBFuzz misses a silent-accept defect under its current templates.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**.

Keywords

Vector database, documentation-implementation consistency testing, test oracle, LLM-derived oracle, LLM-as-judge, source-grounded falsification

1 Introduction

Vector Database Management Systems (VDBMSs) store the embeddings that retrieval-augmented LLM applications depend on. Their defects are costly. An empirical bug study [22] finds that more than half of VDBMS bugs manifest as functional failures, and the VDBMS testing roadmap [19] attributes about 43% of VDBMS bugs to incorrect behavior and identifies oracle definition as a key challenge. Existing VDBMS fuzzers [20] apply classical fuzzing [15] but detect only crashes, so the logical-bug majority goes unreached.

A silent-accept defect does not surface as a crash; it surfaces as wrong results returned to the application. In a retrieval-augmented LLM stack, this means the wrong context reaches the model: a disabled filter returns all matches, a zero-length vector corrupts an index, or a dropped partition hides data the application expects. The downstream model has no signal that its context is corrupted, so the error propagates into generated text without an error code to log. The absence of a crash is the wrong signal for this defect class, which is why an oracle that fires only on crashes (such as VDBFuzz) is structurally blind to it.

We target a prevalent subset: *documentation-implementation defects*, where a VDBMS silently accepts an input or produces a behavior that violates its API documentation. A representative case is Milvus issue #49823, which silently accepts `nprobe=0` although the documentation declares the parameter range as `[1, 16384]`. Such defects corrupt query semantics. They can return data the documentation intended to exclude (for example, a negative score threshold that disables a filter and returns all matches), and most produce no crash.

The oracle problem [3] is acute here. The documented boundary is natural-language prose, so deterministic oracles cannot adjudicate these accept/reject decisions. Differential testing catches mathematical-invariant violations across vendors but cannot adjudicate accept/reject, which diverges across VDBMSs by design [19]. Metamorphic relations [18] address result correctness but not input-acceptance decisions. Property-based testing [4] needs a machine-checkable property and an OpenAPI schema, but VDBMS endpoints rarely expose such schemas. REST-API oracle tools [1, 2, 5] target response-field or runtime properties under per-field structural anchoring (OpenAPI, traces, source), not input-acceptance decisions on system-level prose. We expand this exclusion argument in Section 2 (Table 1); the residual leaves an LLM as the practical oracle.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

Putting an LLM in both the extraction role (documentation to behavioral claims) and the judgment role (response to verdict) creates a false-positive problem with two failure modes. On the extraction side, the LLM hallucinates constraints the documentation does not state [12]. On the judgment side, the same family that extracts a claim tends to confirm it, the self-preference bias established for LLM evaluators [16]. A multi-perspective judging baseline (four specialized judge agents with a vote) raises precision but collapses recall, because the judge agents share the documentation ambiguity. We therefore add a *dev-reviewer* agent that acts as a source-grounded falsifier: it reproduces each candidate, cross-checks it against the implementation source, and tries to disprove it. A candidate that survives all three checks is adjudicated a defect; one that fails any check is suppressed.

We instantiate this as TestVDB, a four-stage pipeline (Section 3): behavioral-claim extraction, test-script generation, sandboxed execution, and defect confirmation, with LLMs automating the documentation-reading, generation, and adjudication steps. TestVDB surfaced 107 candidate issues across three VDBMSs (Milvus, Qdrant, Weaviate); maintainers acknowledged 50 as true-positive defects, and 15 are fixed via merged PR. On a controlled retrospective over 48 maintainer-adjudicated candidates, the dev-reviewer reaches 67% precision and 74% recall (3-run any-confirmed ensemble), against 37% recall without the source anchor. A bidirectional probe against VDBFuzz [20] shows TestVDB reaches a crash-class defect (Qdrant integer overflow on `size= 263`) by contract reasoning, while VDBFuzz misses a TestVDB silent-accept defect (`wait=false` accepting a zero-length vector, #9045) under its current templates.

Contributions.

- **Documentation-implementation defects as a VDBMS target.** We characterize the defect class that VDBMS fuzzers miss, formalize it as consistency with natural-language documentation, and show by an oracle-exclusion argument (Section 2) why an LLM is the practical oracle for the residual.
- **TestVDB.** A four-stage pipeline that uses LLMs to extract behavioral claims from documentation, generate executable tests, run them on sandboxed VDBMSs, and confirm defects by comparing expected and actual behavior (Section 3).
- **Source-grounded falsification.** We diagnose two false-positive failure modes (extraction hallucination, judgment self-preference), show multi-perspective judging is insufficient, and introduce a dev-reviewer agent that falsifies LLM verdicts against implementation source (Sections 4–5).
- **Evaluation.** 107 submitted issues with 50 maintainer-acknowledged true-positive defects (15 merged-PR-fixed) across three VDBMSs; a 48-candidate retrospective showing the source anchor lifts recall from 37% to 74%; and a bidirectional probe against VDBFuzz that explores complementary coverage (Section 6).

2 Background and Problem Setup

VDBMS API documentation, unlike the structured sources that REST-API oracle tools rely on (OpenAPI specifications, execution traces, or source code), is natural-language prose. Constraints are implicit and ambiguous: a parameter described as “optional, default

1” may or may not admit a zero, and the documentation rarely says. This forces a semantic interpretation step that deterministic oracles cannot perform, and an LLM is the scalable proxy for it.

We separate *documentation-implementation consistency* from *correctness*. Consistency asks whether the API’s accept/reject behavior matches its documentation. Correctness asks whether a returned result is mathematically right, such as approximate nearest neighbor (ANN) recall or ranking. TestVDB targets consistency; result correctness of vector search remains open and is not our claim [19].

The oracle problem [3] is acute on this subset. Table 1 walks the standard oracle candidates and the structural reason each misses the documentation-implementation residual. Differential testing catches mathematical-invariant violations across vendors but cannot adjudicate accept/reject, which diverges across VDBMSs by design [19]. Metamorphic relations [18] address result correctness but not input-acceptance decisions, because a metamorphic relation is an output relation, not an input-acceptance transform. Property-based testing [4] needs a machine-checkable property and an OpenAPI schema; VDBMS endpoints serve no schema that encodes these constraints, and several serve no OpenAPI at all. REST-API oracle tools [1, 2, 5] target response-field or runtime properties under per-field structural anchoring, not input-acceptance decisions on system-level prose. The residual leaves an LLM-derived oracle as the practical option. A prior line also derives oracles from documentation with LLMs [7, 14], but keeps the LLM as the final arbiter by validating only through runtime behavior; TestVDB instead falsifies against implementation source (Section 5). The reliability problem this introduces is the subject of Sections 4–5.

3 TestVDB Approach

TestVDB instantiates documentation-implementation consistency testing as a four-stage pipeline: behavioral-claim extraction, test-script generation, sandboxed execution, and defect confirmation, with LLMs automating the documentation-reading, generation, and adjudication steps. A novelty gate finally removes duplicates and known issues. Figure 1 sketches the pipeline.

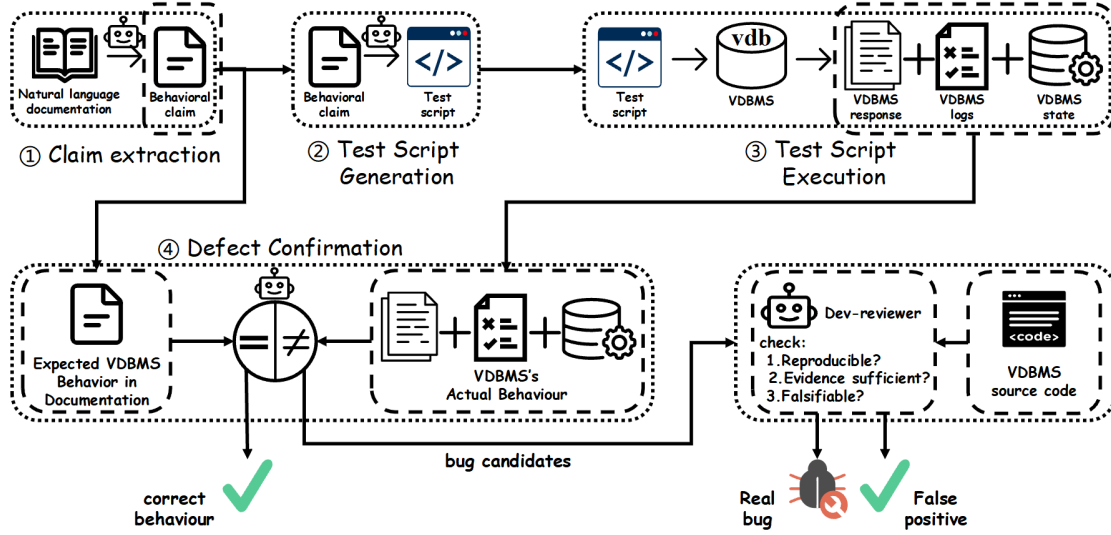
Step 1: behavioral-claim extraction. The contract-formalizer agent reads the VDBMS API documentation and emits structured behavioral claims as a JSON contract. Each claim records a `constraint_id`, the endpoint, the assertion (for example, `nprobe ∈ [1, nlist]`), and a source pointer into the documentation. The LLM performs the semantic interpretation that deterministic oracles cannot, translating “optional, default 1” into a checkable clause.

Step 2: test-script generation. Attack agents turn each claim into an executable probe. For a claim asserting the API rejects `nprobe=0`, the agent generates a request carrying the boundary value and a control value, plus any setup the endpoint needs (collection creation, indexing, payload insertion). Scripts target the boundary so that an accept where the documentation prescribes reject is observable in the response.

Step 3: sandboxed execution. TestVDB runs each script against a Docker-pinned VDBMS instance (Milvus 2.6.19, Qdrant v1.18.2, Weaviate v1.38.2 in our study). Raw HTTP requests and responses

Table 1: Oracle candidates by the defect class they reach, and why the documentation-implementation residual leaves an LLM as the practical oracle. Rows 1 to 5 keep a deterministic oracle; only row 6 enters the LLM regime.

Candidate oracle	Reaches (defect class)	Why it misses the documentation-implementation residual
Crash (VDBFuzz [20])	crash/hang	most documentation-implementation defects do not crash (44 of the 50 true positives)
Differential testing	math invariants across vendors	cross-vendor accept/reject diverges by design; no reference [19]
Metamorphic relations [18]	result correctness (top- k monotonicity, recall vs. ef)	an MR is an output relation; consistency is an input accept/reject decision, and no transform preserves it
Property-based testing [4, 6, 13]	math properties + schema conformance	needs a machine-checkable property and an OpenAPI schema; VDBMS endpoints serve no schema that encodes these constraints, and several serve no OpenAPI
REST doc/spec-derived oracles (AGORA+ [1], SATORI [2], MAS-TOR [5])	response-field/runtime properties from OpenAPI fields, traces, or source	target response-field or runtime properties under per-field anchoring; input-acceptance on system-level prose is out of scope
LLM-derived oracle (TestVDB)	accept/reject vs. API documentation	documentation semantics are not mechanically checkable, leaving an LLM as the practical oracle for the residual

**Figure 1: TestVDB four-stage pipeline. Dashed boxes are LLM-driven; solid boxes (sandboxed execution, dev-reviewer) are not. The dev-reviewer falsifies each candidate defect flagged at Stage 4 against implementation source—the only stage whose ground truth comes from the implementation rather than the documentation.**

are logged for downstream inspection. The sandbox isolates runs so a candidate reproduces under a clean probe, free of tooling artifacts.

Step 4: defect confirmation. A judge agent compares the documented expectation (the claim) against the actual response, the VDBMS state, and the logs. A response that accepts a value the documentation prescribes rejecting becomes a candidate defect. The LLM also adjudicates this step, which introduces the false-positive problem we examine next.

Example: the #49823 path. Milvus documents `nprobe` as an integer in [1, 16384]. The contract-formalizer extracts the claim `nprobe ∈ [1, 16384]` from the documentation passage. An attack agent generates a probe that issues entities/search with

`nprobe=0` and a control with `nprobe=1`. On Milvus 2.6.19 the `nprobe=0` request returns HTTP 200 with results, while the documentation prescribes rejection. The judge agent flags a documentation-implementation violation, and the dev-reviewer verifies that the implementation accepts the value by reading the search-parameter handler in source, which performs no range check on `nprobe`.

LLM automation. TestVDB runs as a multi-agent pipeline [11] on the Claude Code runtime, with each agent defined by a task-structured role prompt and dispatched to a GLM-5.2 backbone under default sampling. The agents span five stage-aligned roles: a contract-formalizer that reads documentation and emits structured JSON claims; four attack agents (boundary, semantic, state,

Table 2: Approximate per-target LLM-call distribution (GLM-5.2).

Stage	Share of calls
Dev-reviewer source-grounding	~50%
Claim extraction + test generation	~25%
Judging + novelty gate	~25%
Total per target	~10 ⁴ calls, ~\$10

discovery) that generate probes from claims; a judge-agent quartet (documentation, evidence, severity, novelty) that triages candidates; the dev-reviewer that falsifies against source; and a reporter that assembles defect reports. Each agent operates independently. For source-grounded falsification, TestVDB clones each target’s source at the pinned version (milvus-src v2.6.19, qdrant-src v1.18.2, weaviate-src v1.38.2); the dev-reviewer greps the clone to trace constants and call chains, and the clone is the only ground truth for what the implementation does. Wall-clock per target is dominated by source-grounding and live Docker re-probes rather than raw LLM latency; a full run is on the order of 10⁴ LLM calls and roughly \$10 per target at current API pricing. Roughly half of the calls are dev-reviewer source-grounding steps (the dominant wall-clock cost), a quarter are claim extraction and test generation, and the rest are judging and the novelty gate (Table 2).

The full prompts (22 agent role definitions), target versions, per-token accounting, the 48-candidate ground truth, and a reproduction driver are in the artifact: <https://github.com/yihui504/testvdb-anon> (prompts under agents/, ground truth under test_questions/, reproduction under reproduction/full52/). All agents run on a GLM-5.2 backbone via the BigModel Anthropic-compatible API under default sampling (no decoding overrides; temperature and top-*p* at provider defaults).

4 The False-Positive Problem

Putting an LLM in both the extraction role and the judgment role creates a false-positive problem with two failure modes. The LLM may extract a claim the documentation does not state, and the LLM may confirm a claim it extracted even when the response contradicts it. The two modes have different causes and require different countermeasures.

Hallucination in extraction. When the contract-formalizer reads ambiguous documentation, it sometimes emits a constraint stronger than the documentation states. A parameter documented as “optional, default 1” becomes “must be at least 1,” or a parameter whose documentation illustrates a power-of-two value becomes “must be a power of two.” Both reject values the documentation allows, so the boundary test reports a violation that is not a defect. This is the test-oracle instance of LLM hallucination [12]: the model supplies a plausible constraint the input does not support.

Self-preference in judgment. When the same LLM family extracts a claim and later judges whether the response conforms, the two roles share biases, and the judge agent tends to confirm the extractor’s claim [16, 21]. Two mechanisms drive this bias: self-recognition (a model recognizes its own family’s outputs [16])

Table 3: Multi-perspective judging baseline: four specialized judge agents voting under a majority-of-four rule.

Judge	Focus
Documentation	Does the response match the extracted clause?
Evidence	Is the response and state evidence sufficient?
Severity	How severe is the defect, if real?
Novelty	Is this a known issue or a new defect?

and perplexity-based familiarity (a model favors lower-perplexity text, which its own outputs tend to be [21]); both predict over-confirmation of same-family claims. A verdict that should be “accept” (the documentation allows the value) is read as “reject” because the judge agent leans toward the extracted clause. Self-preference compounds hallucination: an over-strict extracted claim is more likely to be confirmed by a judge agent of the same family.

Multi-perspective judging is insufficient. A natural counter is to replace the single judge agent with a panel. We tried a multi-perspective design with four specialized judge agents (documentation, evidence, severity, novelty), each reading the documentation independently and voting on each candidate under a majority-of-four rule. On the retrospective (Section 6), it reaches about 80% precision but only about 15% recall. The 15% figure is the highest recall we could reach by tuning the voting threshold; higher thresholds traded recall for at most a few points of precision. The judge agents read the same ambiguous documentation and converge on the same over-strict claim, so voting cannot break the agreement. The failure is structural, not a tuning problem. Any judge agent that reads the documentation shares the documentation’s ambiguity. Table 3 lists the four judge agents and their focus.

The two modes point to different fixes. Hallucination calls for a falsifier that does not depend on the documentation for ground truth, and self-preference calls for a signal that does not come from the same family. The most accessible independent signal is the implementation. The API’s actual behavior is a fact the LLM cannot argue with. This motivates the dev-reviewer.

5 Dev-Reviewer: Source-Grounded Falsifier

The dev-reviewer agent is a source-grounded falsifier. It acts as a VDBMS maintainer would: reproduce each candidate under a clean probe, cross-check it against the implementation source, and try to disprove it. A candidate that survives all three checks is adjudicated a defect; one that fails any check is suppressed as a false positive.

Three checks. For each candidate, the dev-reviewer applies:

- (1) *Independently reproducible.* Does the candidate reproduce under a clean probe, free of tooling artifacts? The dev-reviewer rebuilds a minimal request from the contract and the raw evidence, rather than copying the original probe, and runs it on the live VDBMS.
- (2) *Evidence sufficient.* Do the response, the VDBMS state, and the source code jointly establish the documentation-implementation gap? The dev-reviewer cross-checks the

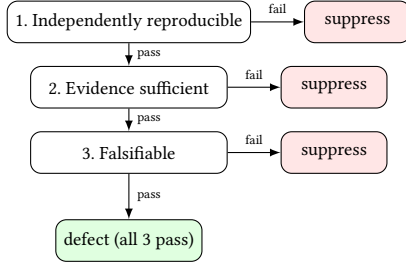


Figure 2: Dev-reviewer three-check falsification. A candidate must pass all three checks to be adjudicated a defect; failing any check suppresses it as a false positive.

response through an independent channel (a primary-key read, a scroll, a count) so the gap does not depend on a single field the request failed to request.

- (3) *Falsifiable*. Can the candidate be disproven by an alternative documented interpretation, such as a by-design behavior or a wont-fix pattern? The dev-reviewer reads the implementation source to check whether the “violation” is an intentional default, an idempotent return, or a maintainer-acknowledged design choice.

Three anchors. The checks draw on three evidence anchors. A *clean-reproduction* anchor re-probes the live API to filter tooling-artifact false positives. A *source-grounded* anchor (the primary falsifier) reads the implementation source across the full clone, tracing constants and call chains, to check whether the constraint is real and whether the source implements it. An optional *threat-model* anchor compares the candidate against maintainer-acknowledged by-design and wont-fix patterns. The same LLM family produces both the behavioral claims and the verdicts; the implementation, an independent information source, is what falsifies them.

Example: the #9255 reversal. Qdrant issue #9255, a payload-filter false positive, illustrates falsification in action. The original oracle flagged a search returning points whose payload appeared absent as a filter violation, but the raw request had not set `with_payload=true`, so the absence was a projection artifact rather than a stored-data gap. The dev-reviewer rebuilt the request with `with_payload=true` and confirmed every returned point carried the filtered field; the candidate was suppressed as a false positive whose root cause was `assertion_depends_on_unrequested_field`. This is the failure mode the multi-perspective panel misses, because every judge agent reads the same response and none re-issues the request.

The dev-reviewer is not a second extractor that reads the documentation again. It reads the source, where the actual behavior is defined, and uses the contract only as the assertion under test. This separation is what breaks self-preference: the falsifier’s ground truth is the implementation, not the same family’s prior claim.

6 Evaluation

We evaluate TestVDB on three VDBMSs (Milvus, Qdrant, Weaviate) and ask three questions:

Table 4: Issues submitted and maintainer-acknowledged per VDBMS.

VDBMS	Submitted	Acknowledged
Milvus	51	22
Weaviate	30	14
Qdrant	26	14
Total	107	50

RQ1 (detection capability): how many documentation-implementation defects does TestVDB surface in real-world VDBMSs?

RQ2 (false-positive suppression): how effectively does the source-grounded dev-reviewer suppress false positives, relative to a single-LLM baseline and a multi-perspective panel?

RQ3 (comparison with VDBFuzz): how many defects detected by TestVDB can the existing crash-oracle approach reach, and vice versa?

RQ1: detection capability. TestVDB surfaced 107 candidate issues across the three systems; maintainers acknowledged 50 as true-positive defects, of which 15 are fixed via merged PR, 16 have open fix-PRs, and 19 are acknowledged but unfixed (8 still open with an accepted label and 11 maintainer-closed as completed without a merged PR). The remaining 57 comprise 22 by-design or rejected and 35 still pending maintainer adjudication. Submissions span Milvus (51), Qdrant (26), and Weaviate (30); true positives span Milvus (22), Qdrant (14), and Weaviate (14). Of the 72 maintainer-adjudicated submissions (50 true-positive, 22 by-design or rejected), the yield precision is 69.4% (Wilson 95% CI [58.0%, 78.9%]); treating all 35 still-pending submissions as false positives gives a worst-case bound of 46.7% (Wilson 95% CI [37.6%, 56.1%]). The 15 merged-PR fixes across three production VDBMSs show practical impact. Table 4 breaks the yield down by vendor. The yield is biased toward documentation-implementation defects by the tool’s design (Section 8); the proportion reflects our yield, not a population estimate. The vendor distribution reflects documentation style: Milvus (22 TP of 51 submitted) concentrates defects in optional-default parameters (`nprobe`, `ef`, `level`) where documentation omits explicit bounds; Qdrant (14 of 26) in search parameters (`timeout`, `score_threshold`) and state operations (`wait=false`); Weaviate (14 of 30) in idempotent behaviors (delete nonexistent class). This mirrors the pattern in Section 4: ambiguous defaults invite over-formalization, while explicit error conditions are uniformly over-formalized.

RQ2: false-positive suppression. On a controlled retrospective over 48 maintainer-adjudicated candidates (27 true-positive, 21 by-design or rejected; Milvus 32, Qdrant 16; Weaviate is covered by a smaller controlled subset (Section 6)), the dev-reviewer (GLM-5.2 backbone, 3-run any-confirmed ensemble) reaches 65% accuracy, 67% precision (Wilson 95% CI [49%, 81%]), and 74% recall, against a single-LLM baseline (no source anchor) of 48%/56%/37%; the source-grounded contract anchor drives the recall gain (37% → 74%). Single runs vary widely: across five independent runs, recall ranges 15–78% (accuracy 44–65%, precision 50–73%, Figure 3), because

Table 5: Dev-reviewer operating points (GLM-5.2). Per-run band = min-max over five runs; any-confirmed = union; majority $\geq 3/5$. The 3-run union is the post-hoc headline. Wilson 95% CIs in brackets (recall/precision values in percent); Bonferroni over the four points ($\alpha=0.05/4$) widens the 3-run CI to roughly [44, 84]/[51, 89], and a bootstrap over the candidate set gives [53, 83]/[71, 96].

Operating point	Accuracy	Precision	Recall
Single run (per-run band)	44–65%	50–73%	15–78%
3-run union (any-confirmed)	65%	67% [49, 81]	74% [55, 87]
5-run union (any-confirmed)	62%	62% [46, 76]	85% [68, 94]
5-run majority (≥ 3 of 5)	N/A	64%	26%

some runs are conservative and confirm few candidates. We report the any-confirmed (union) ensemble as the operating point: the dev-reviewer is a falsifier, so a candidate that survives any independent falsification is more likely a true defect, and under-confirmation is costlier than forwarding a false positive for human triage. Table 5 reports four operating points. Among those that keep precision at least 60%, the 3-run union has the highest recall; the 5-run union reaches higher recall (85%) but lower precision (62%) and lower accuracy (62% vs. 65%), and majority voting collapses recall to 26%. We select the 3-run union as the headline because it sits at the knee of the precision-recall trade-off at modest reproducibility cost. We flag it as a post-hoc, exploratory operating point justified by falsifier semantics, not a pre-registered rule, and the Wilson CIs above do not account for this selection. A Bonferroni correction across the four operating points widens the 3-run CIs to roughly [44%, 84%] (precision) and [51%, 89%] (recall), not changing the qualitative claim. A bootstrap resampling validation (2000 resamples of the 48-candidate set) gives precision 95% CI [53%, 83%] and recall [71%, 96%], supporting that the operating point is not an artifact of the specific candidate sample. A held-out cross-validation (200 random 50/50 splits; select the highest-recall op at precision $\geq 60\%$ on each train half, evaluate on the test half) confirms the selection is not an overfit—3-run, when selected (27% of splits), gives test precision/recall 68%/71% matching the headline, and the held-out-preferred 5-run (61%) reaches test recall 85%; we retain 3-run as a conservative higher-precision headline.

A three-condition ablation on a 12-FP/4-TP control (Milvus v2.6.19, blind) isolates the dev-reviewer’s two anchors (Table 6). Source alone suppresses 9/12 false positives (75%), the threat-model anchor alone 6/12 (50%, unstable on state and concurrency FPs), and their union 11/12 (91%), each retaining all 4 true positives. The source-grounded anchor is the dominant contributor, and the threat-model anchor adds coverage on by-design patterns that source grounding alone reads as implementation choices. The precision gain from the single-LLM baseline (56%) to the dev-reviewer (67%), together with this ablation’s 75% false-positive suppression, directly measures the self-preference reduction [16, 21]: the same-family judge agent over-confirms its own extracted claims, and the source anchor is what breaks that over-confirmation.

To isolate source grounding’s contribution on the full 48-candidate retrospective, we ran the dev-reviewer with Step 3.5 (source grounding) disabled (single run, since this configuration is

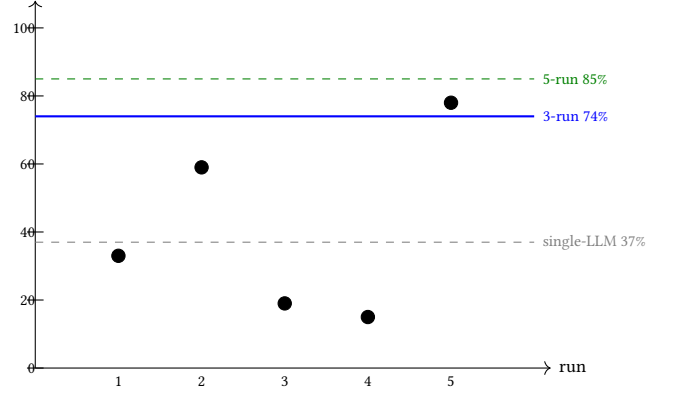


Figure 3: Per-run recall across five independent dev-reviewer runs (dots), against the single-LLM baseline (37%) and any-confirmed ensembles (3-run 74%, 5-run 85%). Single-run recall spans 15–78% because some runs are conservative; the union ensemble absorbs this variance and lifts recall above the baseline.

Table 6: Three-condition ablation on a 12-FP/4-TP Milvus v2.6.19 control (blind). Each configuration retains all 4 true positives. The threat-model anchor is unstable on state and concurrency false positives.

Anchor configuration	FP suppressed	TP retained
Clean-reproduction only	2/12 (17%)	4/4
Source-grounded alone	9/12 (75%)	4/4
Threat-model alone	6/12 (50%)	4/4
Union (source + threat)	11/12 (91%)	4/4

deterministic). Without source access, recall drops from 74% (3-run any-confirmed, the headline) to 19%, and precision from 67% to 56%. The effect is vendor-specific: on Milvus, recall collapses from 80% to 5% (1 of 20 true positives), because parameter validation logic lives in the source rather than the contract or response; on Qdrant, recall is roughly unchanged (57%), because Qdrant’s HTTP responses directly expose accept/reject decisions that the contract already encodes. Source grounding is the dominant contributor to recall, and its value concentrates where the contract under-specifies what the implementation does. Combined with the 12-FP/4-TP source-alone ablation (Table 6, where source alone suppresses 75% of false positives), these two controls triangulate source grounding’s contribution: disabling it collapses recall from 74% to 19%, and enabling it alone accounts for most of the false-positive suppression.

A full independent cross-model re-run on all 48 candidates with three additional families (DeepSeek, Qwen-3.8-Max, and LongCat-2.0, each running the complete SOP with independent live probes and source exploration, three independent runs per family) shows the verdict is family-specific (Cohen’s κ vs. GLM’s 3-run union on a CONFIRMED-vs-not split over $n=48$, where CONFIRMED is the dev-reviewer’s defect verdict: 0.32 DeepSeek, 0.20 LongCat, 0.18 Qwen, all in the slight-to-fair band). On the same 3-run any-confirmed

Table 7: Three configurations on the 48-candidate retrospective (all under the GLM-5.2 backbone). The dev-reviewer with source grounding is the only configuration that combines useful precision with useful recall; the other two trade one for the other. The dev-reviewer row is the 3-run any-confirmed union; Table 5 reports its full operating-point range.

Configuration	Accuracy	Precision	Recall
Single-LLM (no source)	48%	56%	37%
Multi-perspective (4 judge agents)	50%	80%	15%
Dev-reviewer (source-grounded)	65%	67% [49, 81]	74% [55, 87]

union, combined recall is 56% (DeepSeek), 22% (LongCat), and 19% (Qwen) vs. 74% for GLM-5.2’s 3-run union (85% at its 5-run union, an upper bound). The gap is vendor-specific: DeepSeek reaches 75% recall on Milvus (3-run union, inter-run $\kappa = 1.0$), approaching GLM-5.2’s Milvus 95% under the same protocol, but collapses on Qdrant (0% vs. GLM’s 29%), and all three families stay at $\leq 14\%$ recall on Qdrant. Inter-run self-consistency is high for DeepSeek/LongCat, lower for Qwen ($\kappa = 0.65$). The Qwen variance on Milvus is structured: all six run-to-run disagreements are contract-vs-source tensions (over-strict assertions colliding with by-design source behavior); tightening the contract-formalizer would raise κ sharply. Source-grounded falsification is backbone-dependent, and the dependence concentrates on Qdrant. A cross-family voting probe reaches 81% recall at 73% precision (GLM $\cup$ DeepSeek; families complementary, $\kappa = 0.39$); cross-family generalization remains open, but voting is a viable mitigation direction. Multi-perspective judging (Section 4) reaches about 80% precision but only about 15% recall on the same candidates. Per vendor, the dev-reviewer reaches Milvus 69%/73%/80% (accuracy/precision/recall) and Qdrant 56%/50%/57%; the single-LLM baseline reaches Milvus 48%/60%/40% and Qdrant 48%/50%/30%. Table 7 compares the three configurations on the 48-candidate retrospective.

RQ3: comparison with VDBFuzz. We ran a bidirectional reachability probe between TestVDB and VDBFuzz [20] on Qdrant (Table 8). The systematic direction (a fixed-budget run, not an $n=1$ case) is VDBFuzz against TestVDB’s pinned version: on Qdrant v1.18.2, where all 14 of TestVDB’s Qdrant true positives are live, VDBFuzz (default configuration) ran over 26,000 mutated requests—the crash baseline on v1.18.2—and reached 0 of them. This is structural rather than a budget artifact: the 14 are silent accepts that return HTTP 200, and a crash oracle cannot detect a non-crashing accept by construction. The two Qdrant crashes VDBFuzz reports in its own case study are also fixed in v1.18.2, so the run cleanly separates the two oracles’ reach on this version. On the older Qdrant v1.4.0, where VDBFuzz’s integer-overflow crash on a vector size of 2^{63} is still live, TestVDB’s boundary probe reaches it as a documentation-implementation violation: the OpenAPI declares $\text{size} \geq 1$ with no maximum, so the value is documented-valid yet the implementation panics. On Qdrant v1.18.0, where TestVDB’s acknowledged empty-vector defect (#9045, $\text{wait}=\text{false}$ silently accepts a zero-length vector) is live, VDBFuzz’s template suite reports no failure: every points-upsert template uses $\text{wait}=\text{true}$ (the path that correctly rejects) and does not probe $\text{wait}=\text{false}$. We read the reverse

Table 8: Bidirectional reachability between TestVDB and VDBFuzz on Qdrant. The systematic direction is VDBFuzz on v1.18.2 (TestVDB’s pinned version, 14 TPs live, 26,000 requests, 0 reached); the controlled cases on v1.4.0 and v1.18.0 isolate crash-class mechanisms ($n=1$ each).

Version	Reproduces	Fix state
v1.4.0	VDBFuzz’s integer-overflow crash (size= 2^{63})	fixed in v1.5.0
v1.18.0	TestVDB’s #9045 ($\text{wait}=\text{false}$ accepts a zero-length vector); #7967 panic under load	fixed May 2026
v1.18.2	— (TestVDB’s target version)	both fixed

direction as a limitation of VDBFuzz’s current templates and input coverage, not a property of crash oracles as a class. The two controlled cases below are $n=1$ each and isolate the mechanism on crash-class defects; the systematic direction above (26,000 requests, 0 of 14 silent-accept true positives reached) is the generalizable one.

The #9045 root cause shows why the residual survives crash-focused patching: vector validation runs only under a `debug_assert` that release builds skip, so $\text{wait}=\text{false}$ accepts a zero-length vector while $\text{wait}=\text{true}$ rejects it. The resulting panic (#7967) survived three crash-fix iterations across v1.16.3–v1.18.0 until an API-level check independent of `wait` resolved it; a crash oracle would flag the panic once triggered, but the silent-accept residual on $\text{wait}=\text{false}$ stays out of reach.

Threats to validity. Internal validity. Single-run recall varies widely (15–78%) because the LLM judge agent is high-variance; the any-confirmed union ensemble is the operating point we match to this variance, since majority voting collapses recall when conservative runs dominate the vote. The operating point is post-hoc, however, and the Wilson CIs in Table 5 do not account for selection across the four operating points. The 48-candidate retrospective is maintainer-adjudicated but non-random, and we did not pre-register the candidate set; a capture-recapture or unbiased defect-sample estimation is future work.

External validity. The controlled retrospective covers Milvus and Qdrant; to probe a third VDBMS we ran the dev-reviewer once on a Weaviate controlled subset of 11 candidates (v1.38.0 evidence re-probed against v1.38.2, source-grounded against the v1.38.2 clone). It produced 7 source-grounded confirmations and 4 false-positives, overturning five stage2 decisions and agreeing with the maintainer on both fixed-by-PR cases (#11981, #11401); Weaviate’s sparse maintainer-fixed TP density (3 of 30) precludes a per-vendor recall headline. Result correctness of vector search (ANN recall, ranking) is out of scope. Transfer to documentation regimes beyond VDBMSs (REST APIs without OpenAPI, configuration validation, policy-as-code) is claimed on structural grounds only and is not empirically tested; even one non-VDBMS case study would strengthen the claim.

Construct validity. All dev-reviewer results use a single LLM family (GLM-5.2). A full independent cross-model re-run on all 48 candidates (DeepSeek, Qwen-3.8-Max, and LongCat-2.0, each running the complete SOP) shows the verdict is family-specific

(Cohen’s κ vs. GLM’s 3-run union: 0.32 DeepSeek, 0.20 LongCat, 0.18 Qwen), so we cannot claim cross-family robustness. We do not estimate recall because no public ground-truth defect catalog for VDBMSs exists. The implementation-as-correct assumption (Section 8) bounds the approach: an implementation bug can wrongly falsify a clause whose documentation is right. In practice, 31 of the 50 true positives have merged or open fix-PRs (implementation confirmed buggy by maintainers), and no false positive was traced to an implementation bug—all FPs are maintainer-adjudicated by-design—so the assumption produced no observed false negatives in the retrospective.

7 Related Work

VDBMS testing. VDBFuzz [20] is the first dedicated VDBMS fuzzer and uses crash as its oracle. Its template-based input mutation and crash detection reach exactly the crash-class subset that documentation-implementation defects, which silently accept rather than crash, escape; the bidirectional probe in Section 6 makes this complementarity concrete. The roadmap [19] sets the agenda we build on, identifying oracle definition as a key challenge for VDBMS testing and flagging differential testing as hard in this setting. The empirical bug study [22] provides the taxonomy that motivates our focus on logical bugs, attributing more than half of VDBMS defects to functional failures rather than crashes.

REST-API oracle generation. A recent line derives test oracles for REST APIs from structured sources: AGORA+ [1] from execution traces, SATORI [2] from OpenAPI specifications, and MASTOR [5] from source code. AGORA+ infers invariants from observed traffic and so cannot reach inputs the traffic did not exercise. SATORI uses an LLM to infer response-field oracles (e.g., enum values, length, range) from OpenAPI field names, types, and natural-language descriptions; its oracle catalog targets response-field properties, not input-acceptance decisions. MASTOR, the closest to this work, reads source to generate oracles that encode implemented behavior and so cannot detect a gap between documentation and code; TestVDB reads source as a falsifier of documentation-derived claims and targets exactly that gap. All three operate under per-field structural anchoring (OpenAPI fields, execution traces, or source) and target response-field or runtime properties; none adjudicates input-acceptance decisions on system-level prose. A concrete boundary: SATORI’s oracle catalog (format, length, enum, range of response fields) is anchored on per-field OpenAPI properties, and the input-acceptance decision is outside its oracle target; VDBMS input-acceptance constraints, in addition, live in system-level prose with no per-field schema to anchor on.

LLM-as-judge reliability. Self-preference bias, in which an LLM evaluator favors its own family’s outputs, is established for general text evaluation [16, 21]. Panickssery et al. show that self-recognition correlates with self-preference across model families, and we observe the analogous bias in the test-oracle pipeline, where the same family extracts a claim and later judges conformance. A separate reliability axis is intra-judge self-inconsistency: Haldar et al. [8] show that a single LLM judge agent’s ratings on the same input vary across runs, a sampling-noise phenomenon in the judgment step. Our false-positive problem intersects both, with hallucination

at extraction [12] added: an LLM may stably produce a wrong claim across runs and families when the documentation is ambiguous. Source-grounded falsification mitigates all three by moving the ground truth from the LLM to the implementation.

Documentation-derived oracles. Toradocu [7] pioneered NL-documentation oracle extraction (Javadoc @throws via NLP), with acknowledged false positives. Doc2OracLL [10] extended this to LLMs; AugmenTest [14] infers oracles from available documentation; ChatAssert [9] repairs false positives via compilation/execution feedback; Testora [17] uses PR descriptions as a regression oracle (~55% precision). All treat the LLM as the final semantic arbiter, verifying through runtime behavior; TestVDB instead falsifies against implementation source, targeting the ambiguous-prose regime Javadoc and OpenAPI extraction do not enter.

8 Discussion and Limitations

The setting is not specific to VDBMSs in principle, though our evaluation is VDBMS-only. Systems whose documentation is natural-language prose rather than a structured specification, such as REST APIs without OpenAPI coverage, configuration validation, and policy-as-code checks, enter the same regime. We claim transferability only to structurally similar documentation regimes, and only as a preliminary portability probe rather than a generalization result; to probe it, we ran the pipeline on Apache CouchDB 3.4.3 (all three gaps reproduced on 3.5.2; the limit-handling code is byte-identical in the 3.5.x source) and Elasticsearch 8.11.0. Elasticsearch rejected every invalid probe with a 400 error. CouchDB’s `/_all_docs` behaved the same, but its `/_changes` feed exposed three input-validation gaps: `limit=0` returns zero rows although the docs state “the same effect as 1”; non-numeric limit returns HTTP 500 (badarg) rather than the 400 that `/_all_docs` returns; and `limit=-1` is silently accepted as unlimited. Two of the three gaps were submitted to the CouchDB maintainers (apache/couchdb #6087, #6088) and acknowledged as bugs (labeled bug, closed within one day), confirming that these are genuine documentation-implementation defects rather than by-design behavior. The gaps concentrate where the contract is a prose remark rather than a schema-enforced bound—the same regime behind the 50 maintainer-acknowledged VDBMS defects—and the pipeline ports across non-VDBMS targets.

Beyond the threats in Section 6, two limitations bound the approach. Source-grounded falsification requires source, so it does not transfer to closed-source VDBMSs, and it treats the implementation as correct, so an implementation bug can wrongly falsify a clause whose documentation is right. The 15 merged-PR fixes suggest the implementation-as-correct assumption holds often enough to be useful, without guarantee. We did not observe maintainer-rejected confirmed TPs where the documentation itself was wrong; such cases would surface as false negatives of the dev-reviewer and are most likely where a maintainer closes a confirmed candidate as wont-fix without a source-level explanation. We did not quantify this false-negative rate, which would require a ground-truth catalog of documentation errors that does not exist for VDBMSs.

Our 107-submission yield is biased by the tool's design toward documentation-implementation defects, so the proportion we observe is not an estimate of the distribution in VDBMSs at large. A full-space estimation with capture-recapture or an unbiased defect sample is future work.

9 Conclusion

We target documentation-implementation defects in Vector Database Management Systems, where a VDBMS silently accepts inputs that violate its API documentation. Because the documented boundary is natural-language prose, current instantiations of deterministic oracles miss these accept/reject decisions, leaving an LLM as the practical oracle. The LLM introduces two false-positive failure modes, hallucination in claim extraction and self-preference in judgment, that multi-perspective judging cannot resolve. Our dev-reviewer agent reads implementation source as an independent falsifier and breaks both modes. TestVDB surfaced 107 candidate issues across three VDBMSs with 50 maintainer-acknowledged true-positive defects (15 merged-PR-fixed), reaches 67% precision and 74% recall on a controlled retrospective against 37% recall without source grounding, and a bidirectional probe against VDBFuzz explores complementary coverage. The boundary we draw, between problems where structured-source extraction suffices and those where natural-language documentation forces LLM interpretation, marks where an LLM-derived oracle needs an independent falsifier.

References

- [1] Juan C. Alonso, Michael D. Ernst, Sergio Segura, and Antonio Ruiz-Cortés. 2025. Test Oracle Generation for REST APIs. *ACM Transactions on Software Engineering and Methodology (TOSEM)* (2025). doi:10.1145/3726524
- [2] Juan C. Alonso, Alberto Martin-Lopez, Sergio Segura, Gabriele Bavota, and Antonio Ruiz-Cortés. 2025. SATORI: Static Test Oracle Generation for REST APIs. In *Proc. of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. arXiv:2508.16318.
- [3] Earl T. Barr, Mark Harman, Yue Jia, Alexandru Marginean, and Justyna Petke. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 7 (2015).
- [4] Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the ACM SIGPLAN International Conference on Functional Programming (ICFP)*.
- [5] Sida Deng, Rubing Huang, Zhenzhen Yang, Man Zhang, Xuan Xie, and Rongcun Wang. 2026. MASTOR: A Multi-Agent Approach to Semantic Test Oracle Generation for RESTful APIs. arXiv:2606.10465.
- [6] Dmitry Dygalo. 2024. Schemathesis: Property-Based API Testing for OpenAPI and GraphQL. <https://schemathesis.io>. Accessed: 2026-07.
- [7] Alberto Goffi, Alessandra Gorla, Michael D. Ernst, and Mauro Pezzè. 2016. Automatic Generation of Oracles for Exceptional Behaviors. In *Proc. of the 25th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 213–224. doi:10.1145/2931037.2931062
- [8] Reshma Halder et al. 2025. Rating Roulette: Self-Inconsistency in LLM-As-A-Judge Frameworks. In *Findings of the Association for Computational Linguistics: EMNLP*.
- [9] Ishrak Hayet and Andrew Scott. 2025. ChatAssert: LLM-Based Test Oracle Generation With External Tools Assistance. *IEEE Transactions on Software Engineering* 51, 1 (2025), 305–319. doi:10.1109/TSE.2024.3519159
- [10] Soneya Binta Hossain, Raygan Taylor, and Matthew B. Dwyer. 2025. Doc2OracLL: Investigating the Impact of Documentation on LLM-Based Test Oracle Generation. In *Proc. of the ACM International Conference on the Foundations of Software Engineering (FSE)*. doi:10.1145/3729354 arXiv:2412.09360.
- [11] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 33, 5 (2024), 220:1–220:79. doi:10.1145/3695988
- [12] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. *Comput. Surveys* 55, 12 (2023), 248:1–248:38. doi:10.1145/3571730
- [13] Martin Karlsson, Amin Sabet, and Michael Pradel. 2020. QuickREST: Property-based Testing of REST APIs. In *Proc. IEEE Int. Conf. on Software Testing, Verification and Validation (ICST)*.
- [14] Shaker Mahmud Khandaker, Fitsum Kifetew, Davide Prandi, and Angelo Susi. 2025. AugmentTest: Enhancing Tests with LLM-Driven Oracles. arXiv:2501.17461.
- [15] Valentin J. M. Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. 2021. The Art, Science, and Engineering of Fuzzing: A Survey. *IEEE Transactions on Software Engineering* 47, 11 (2021), 2312–2331. doi:10.1109/TSE.2019.2946563
- [16] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. 2024. LLM Evaluators Recognize and Favor Their Own Generations. arXiv:2404.13076.
- [17] Michael Pradel. 2026. Using Natural Language Intent to Detect Behavioral Regressions. In *Proc. of the 48th IEEE/ACM International Conference on Software Engineering (ICSE)*. arXiv:2503.18597.
- [18] Guanyu Wang, Yuekang Li, Yi Liu, Gelei Deng, Tianlin Li, Guosheng Xu, Yang Liu, Haoyu Wang, and Kailong Wang. 2024. MeTMAP: Metamorphic Testing for Detecting False Vector Matching Problems in LLM Augmented Generation. In *Proceedings of the IEEE/ACM International Conference on AI Foundation Models and Software Engineering (FORGE)*.
- [19] Shenao Wang, Yanjie Zhao, Yinglin Xie, Zhao Liu, Xinyi Hou, Quanchen Zou, and Haoyu Wang. 2025. Towards Reliable Vector Database Management Systems: A Software Testing Roadmap for 2030. arXiv preprint arXiv:2502.20812.
- [20] Shenao Wang, Yanjie Zhao, Yinglin Xie, Zhao Liu, Xinyi Hou, Quanchen Zou, and Haoyu Wang. 2026. VDBFuzz: Understanding and Detecting Crash Bugs in Vector Database Management Systems. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*.
- [21] Kohei Wataoka and Ryo Takahashi. 2024. Self-Preference Bias in LLM-as-a-Judge. arXiv:2410.21819.
- [22] Yinglin Xie, Xinyi Hou, Yanjie Zhao, Shenao Wang, Kai Chen, and Haoyu Wang. 2025. Toward Understanding Bugs in Vector Database Management Systems. arXiv preprint arXiv:2506.02617.

A Key agent prompts

This appendix excerpts the role prompts for the two agents that most affect the pipeline's output: the contract-formalizer (extraction) and the dev-reviewer (falsification). The full 22 agent definitions are in the artifact (<https://github.com/yihui504/testvdb-anon> under agents/).

Contract-formalizer (extraction).

Role: convert the natural-language API knowledge in `raw_knowledge.md` into a structured JSON contract. **Input:** `raw_knowledge.md`. **Output:** `structured_contract.json`, where each claim records a `constraint_id`, the endpoint, the assertion (for example, `nprobe ∈ [1,nlist]`), and a source pointer into the documentation. No network access; all document content is local. The agent must not invent constraints the documentation does not state—the test-oracle instance of hallucination [12] that Section 4 addresses.

Dev-reviewer (falsification).

Role: developer-perspective final review (double-blind). **Data access:** `verified_only`—raw output_*.log, `structured_contract.json`, and the implementation source clone; the dev-reviewer must not read other judge agents' votes or rationale. **Source grounding:** Grep the local source clone to trace constants and call chains. **Three checks per candidate:** (1) *independently reproducible*—rebuild a minimal request from the contract and run it on the live VDBMS; (2) *evidence sufficient*—cross-check the response through an independent channel; (3) *falsifiable*—read the source to check whether the “violation” is an intentional default, an idempotent return, or a maintainer-acknowledged design choice. A candidate surviving all three is adjudicated a defect; one failing any check is suppressed as a false positive.